

Assignment 3. Deep Learning in Data Science.

Author:
Rodrigo Castellano Ontiveros
roco@kth.se

Contents

1	Assignment 3 exercises	1
1.1	Gradients	1
1.2	3-layer NN	2
1.3	9-layer NN	3
1.4	Coarse search	3
1.5	Fine search	3
1.6	Sensibility to initialization	3
2	Bonus points	4
2.1	BN over fixed model	4
2.2	Adam and learning rate decay	5
2.3	Higher number of nodes in each layer	6

1 Assignment 3 exercises

1.1 Gradients

For this assignment, I made the changes needed to the code of the previous assignment in order to adapt it to k-hidden layers. To check the gradients, I adapted the functions to check the analytical gradients. Due to computational time, I tried with [50] and [50,50] hidden layers, 10 dimensions and 5 images. The results for 2 layers are shown in table 1. For other number of layers the results were very similar. In most of the cases the relative differences are very small, which indicates that the gradients are properly computed.

	Gradient 1	Gradient 2
W	2.243e-08	9.319e-09
b	8.881e-3	5.213e-09
Gamma	9.902e-09	
Beta	7.156e-09	

Table 1: Relative difference of gradients for two layers.

1.2 3-layer NN

For a NN with 3 layers and the settings specified in the assignment, the results are shown in figures 1,2. One difference is that I applied decay of the maximum learning rate every cycle.

For batch normalization (BN) it can be seen that the loss function is even less smooth than the original, even though the performance was better. This is probably because the effect of BN is more noticeable when there is a large number of layers.

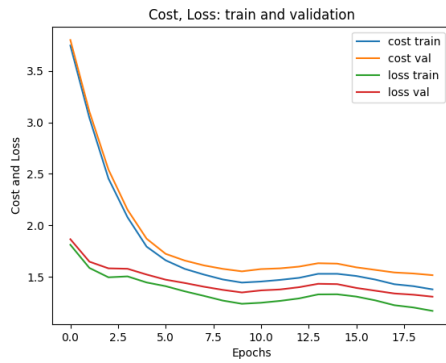


Figure 1: Cost and loss for the 3-layers NN. No batch normalization used.

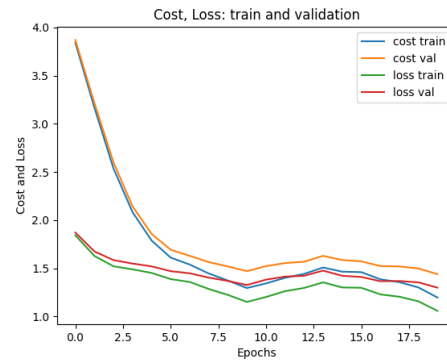


Figure 2: Cost and loss for the 3-layers NN. Batch normalization used.

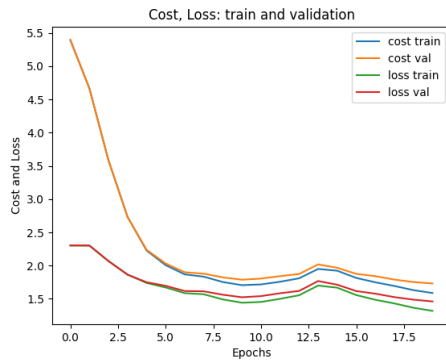


Figure 3: Cost and loss for the 9-layers NN. No batch normalization used.

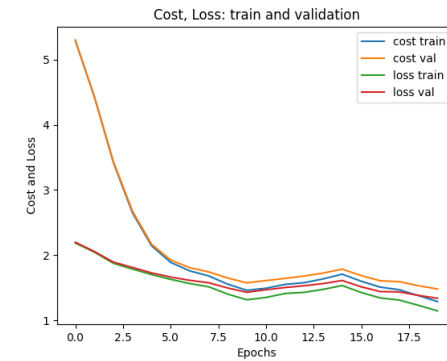


Figure 4: Cost and loss for the 9-layers NN. Batch normalization used.

1.3 9-layer NN

For a NN with 9 layers and the settings specified in the assignment, the results are shown in 3,4. The maximum learning rate was reduced every cycle. This time it is more clear how the BN makes the training smoother, also leading to noticeable increase in performance.

1.4 Coarse search

For the 3-layer NN, the values of λ in coarse search were

$$\lambda = [0.000407, 0.00418, 0.098633, 0.004114, 5.1e - 05, 0.002024, 0.000479, 0.02255]$$

The highest validation accuracy was 53.3% for $\lambda = 0.004114$ with a test accuracy of 52.9%. The second best validation accuracy was 52.1% for $\lambda = 0.002024$ with a test accuracy of 53%.

Nevertheless, the best result was obtained with the configuration mentioned in assignment 3, $\lambda = 0.005$, with a validation accuracy of 53.7% and a test accuracy of 53.6%.

1.5 Fine search

Due to the obtained accuracies, the fine search was done with the values

$$\lambda = [0.00396, 0.002212, 0.001003, 0.00222, 0.006655, 0.002651, 0.0038, 0.001451]$$

The highest validation accuracy was 53.7% for $\lambda = 0.006655$ with a test accuracy of 53.7%. The second best validation accuracy was 53.2% for $\lambda = 0.00396$ with a test accuracy of 53%. The third best validation accuracy was 53.2% for $\lambda = 0.002212$ with a test accuracy of 52.5%.

A training for 3 cycles was done with the best setting, and the validation accuracy was 53.8% for $\lambda = 0.006655$ with a test accuracy of 53.6%.

1.6 Sensibility to initialization

Training was done for with and without BN for $\sigma = [1e - 1, 1e - 3, 1e - 4]$ in the initialization of the weights. The results are shown in table 2. The results for BN are stable and higher than the case where no BN is used (from a certain σ on the accuracy decreases up to 10%). In figures 5-14 it is shown the cost and loss for all the cases. The conclusion is that BN is robust to weight initialization, and where there is no weight initialization changes in the initialization of the weights can lead to bad performance due to, for example, exploding gradients or the contrary effect. Also we can appreciate that the accuracy with BN is mostly constant and higher than when there is no BN.

Sigma	BN	Val accuracy(%)	Test accuracy(%)
1e-1	Yes	52.1	52
1e-1	No	50.1	50.8
1e-3	Yes	52.2	52.9
1e-3	No	10.3	10.1
1e-4	Yes	52.4	52.7
1e-4	No	10.3	10.1

Table 2: Results of the trainings for the sensitivity to initialization.

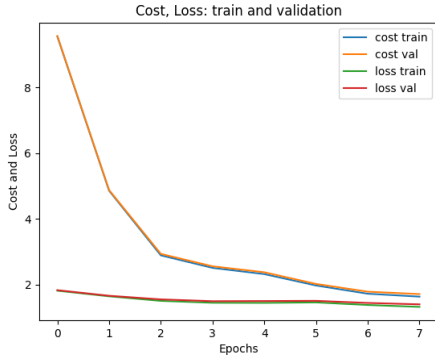


Figure 5: Cost and loss for no batch normalization and $\sigma = 1e - 1$.

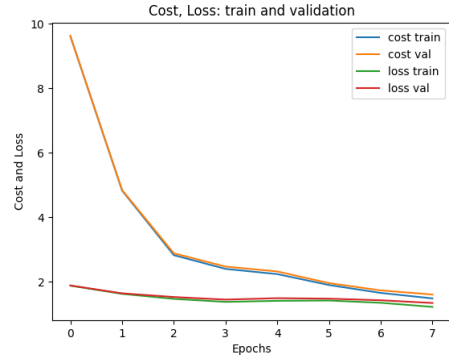


Figure 6: Cost and loss for batch normalization and $\sigma = 1e - 1$.

2 Bonus points

2.1 BN over fixed model

I changed the equations of how to compute μ_{av} and v_{av} so that they are computed over the whole training set. That was doing by calculating the expectations. For example, for μ_{av} , in every layer the new $\mu_{av}^{(l)}$ is calculated as the expected value of all the $\mu_{av}^{(l)}$ obtained before (the expected value). To avoid storing all the previous $\mu_{av}^{(l)}$, the expression applied was the following

$$\mu_X = \mathbb{E}[X] = \frac{1}{n} \sum_{i=1}^n X_i$$

For a new element,

$$\mu_{X_{n+1}} = \frac{\mu_{X_n}}{n} + \frac{n}{n+1} X_{n+1}$$

where X is a random variable with n elements, and then a new element is included. This is more practical because allows to not keep track of all the values in X .

The obtained validation accuracy was 52.8% for the validation test and 52.6% for the test set (). The result is a bit lower but the implementation seems to be fine. BN was also applied to the test set with these expressions already computed. The cost and loss function can be seen in figure 11.

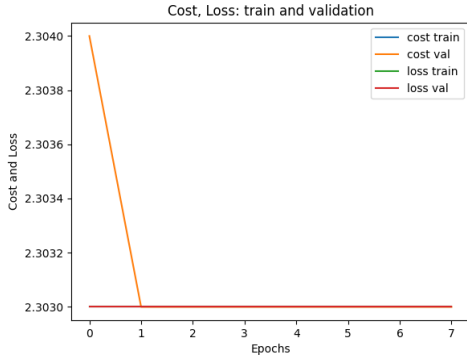


Figure 7: Cost and loss for no batch normalization and $\sigma = 1e - 3$.

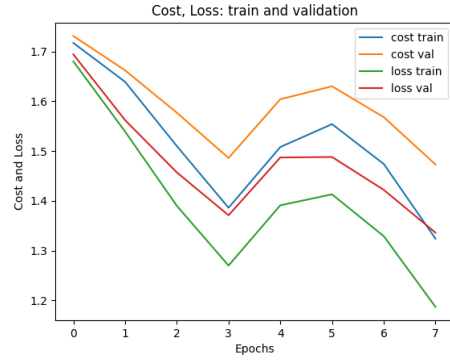


Figure 8: Cost and loss for batch normalization and $\sigma = 1e - 3$.

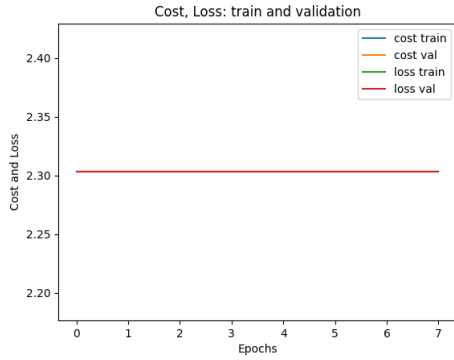


Figure 9: Cost and loss for no batch normalization and $\sigma = 1e - 4$.

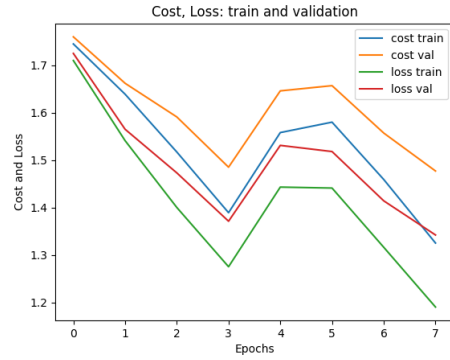


Figure 10: Cost and loss for batch normalization and $\sigma = 1e - 4$.

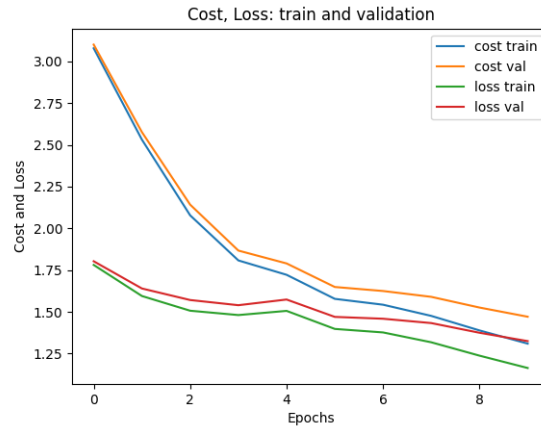


Figure 11: Cost and loss for BN over the train set.

2.2 Adam and learning rate decay

Adam optimizer was implemented. As before, the training was done with 3-layers and the cyclic learning rate and decay were used. For cyclic learning rate the results were

very bad, but for decay the results were better. For a total o 20 epochs and a learning rate decay of 0.9, the validation accuracy was 52.7% and the test accuracy was 51.2%. The loss and cost function are shown in figure 12. The results are very similar, even a bit worse than those with SGD and cyclic learning rate. Despite that, further training and tuning could have lead to better results.

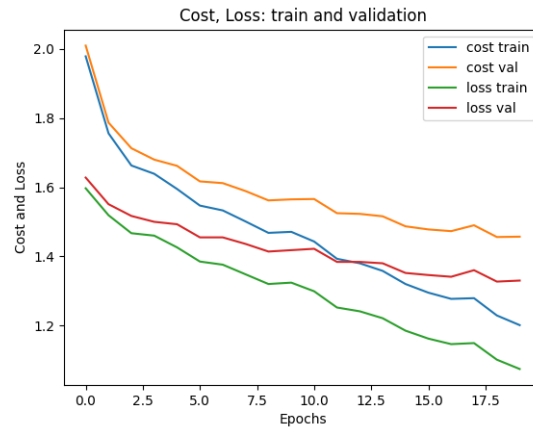


Figure 12: Cost and loss for Adam optimizer.

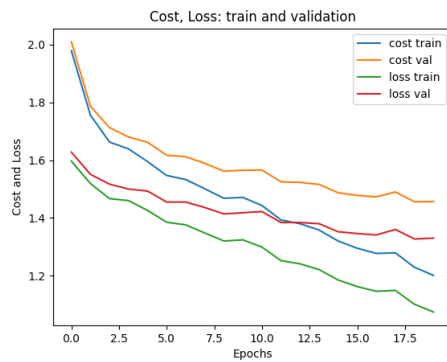


Figure 13: Cost and loss for Adam optimizer.

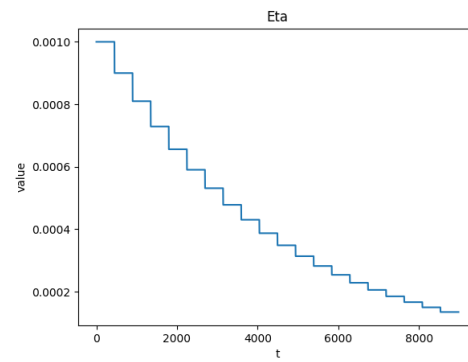


Figure 14: Learning rate for Adam optimizer.

2.3 Higher number of nodes in each layer

The last experiment was the one that lead to the best results. First, the number of nodes was set to 150 per layer, and later it was set to 300 nodes per layer, which gave the best result. For SGD with a cyclic learning rate (2 cycles, 20 epochs in total), the validation accuracy was 58.9%, and the test accuracy 57-7%. The loss and cost function are shown in figure 15.

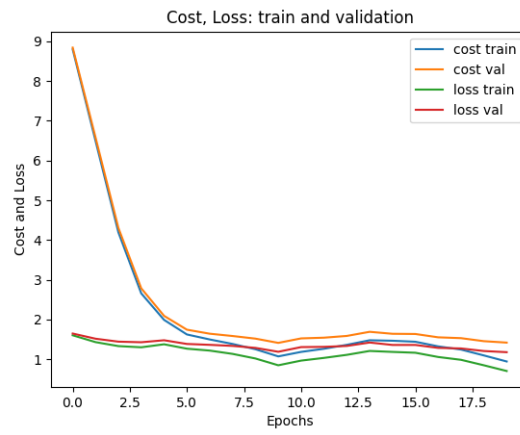


Figure 15: Cost and loss for 300 hidden nodes in each layer.